

LES WEB SERVICES ET LA BASE MONGODB

**DEVELOPPEMENT AVEC ECLIPSE JAVA EE
ET UTILISATION DES PLUGINS SOAPUI ET MONJADB**

Table des matières

INTRODUCTION.....	3
PREMIERE PARTIE	4
1 L'intérêt des Web Services	4
2 Le protocole de communication SOAP	4
3 Le WSDL (Web Services Description Language)	5
4 Structure d'un WSDL	6
5 Principe d'un WSDL	8
6 Tester un WSDL.....	10
7 Tester un Web Service avec le Plugin SoapUI	11
DEUXIEME PARTIE	13
1 Schéma d'ensemble du projet	13
2 Créer un Web Service avec Eclipse Java EE.....	14
3 Se connecter à une base MongoDB et ajouter des enregistrements dans une collection	20
4 Visualiser et modifier une base MongoDB avec le plugin MonjaDB.....	21
5 Créer des mises à jour avec Java Mongo Driver	22
CONCLUSION.....	26
Bibliographie	27
Outils et technologies utilisés	27

INTRODUCTION

Le but du projet est d'appréhender les Web Services dans un environnement de gestion de base de données MongoDB.

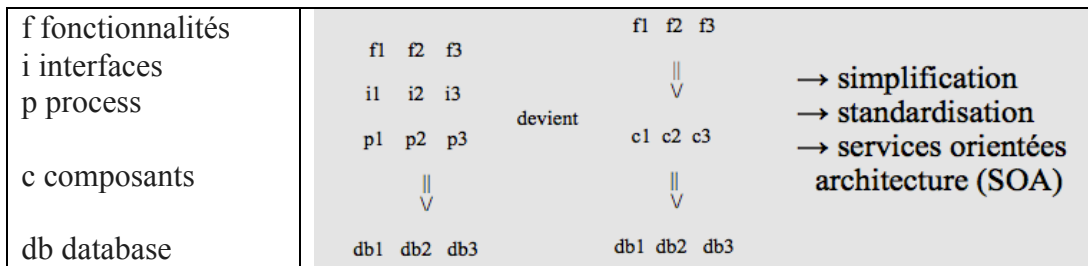
Nous allons vous présenter dans une première partie le niveau conceptuel des Web Services ; le protocole de communication SOAP, le langage de description des Web Services (WSDL) et un exemple d'intégration d'un WSDL appelant un Web Service sur internet avec le logiciel SoapUI.

Dans une deuxième partie nous vous présenterons en début de cette partie un schéma d'ensemble qui est notre retour d'expérience, construit progressivement au cours de notre étude et ensuite nous ferons une présentation de chaque étape qui permet de concevoir un Web Service de bout en bout ; La création d'un Web service simple et son test en local avec le logiciel SoapUI, la connexion, la visualisation et la modification d'une base de données MongoDB et son utilisation avec une librairie Java Mongo Driver.

PREMIERE PARTIE

1 L'intérêt des Web Services

Chaque fonctionnalité peut accéder à une base de données via des process/interfaces spécifiques
Des services métiers réutilisables sont développés et donnent accès à des process et à des bases de données et cela quelque soit la technologie = 'principe du Web' utilisée par la fonctionnalité et par l'interface.



2 Le protocole de communication SOAP

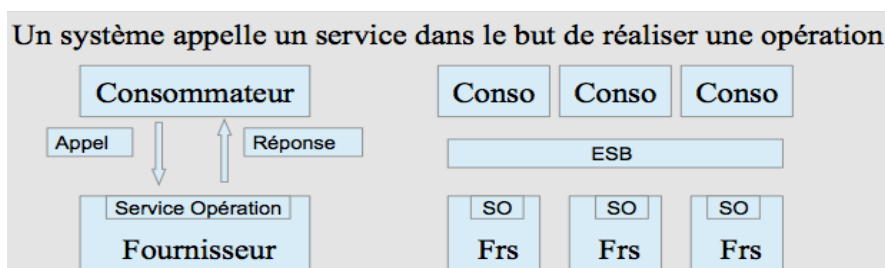
SOAP (acronyme de Simple Object Access Protocol ou Service Oriented Architecture) est un protocole de RPC orienté objet bâti sur XML et constitue une norme pour communiquer du contenu.

Il permet la transmission de messages entre objets distants, ce qui veut dire qu'il autorise un objet à invoquer des méthodes d'objets physiquement situés sur un autre serveur. Le transfert se fait le plus souvent à l'aide du protocole HTTP, mais peut également se faire avec SMTP.

Le protocole SOAP est composé de deux parties :

- une enveloppe, contenant des informations sur le message lui-même afin de permettre son acheminement et son traitement, éventuellement seront présentes les éléments de déclaration d'un ESB (Bus de services qui permet d'authentifier les accès des consommateurs/clients, de contrôler la conformité, de participer au routage des requêtes et à la supervision des flux),
- un modèle de données, définissant le format du message, c'est-à-dire les informations à transmettre.

Le RPC (Remote Procedure Call) est initié par le Consommateur / client qui envoie un message de requête à un Fournisseur / serveur distant connu pour exécuter une procédure spécifique avec des paramètres spécifiques. Le serveur distant envoie une réponse au client et l'application continue son déroulement.



3 Le WSDL (Web Services Description Language)

Le WSDL est un langage de description de service Web.

Le langage WSDL permet la génération de proxy pour les services Web de façon totalement indépendante de la plate-forme.

Un document WSDL traduit un contrat de service entre un consommateur/client et un fournisseur/serveur du service.

Le WSDL (Web Services Description Language) « Whiz-Deul » sert à décrire :

- le protocole de communication (SOAP RPC ou SOAP orienté message,
- le format de messages requis pour communiquer avec ce service,
- les méthodes (au sens Java) que le client peut invoquer,
- la localisation du service.

Le document WSDL est écrit en XML.

Un schéma lui est associé.

Des fichiers XSD définissent les types de données.

XSD définit de façon structurée le type de contenu, la syntaxe et la sémantique d'un document XML. Il est également utilisé pour valider un document XML, c'est à dire vérifier si le document XML respecte les règles décrites dans le document XSD.

Une description WSDL est un document XML qui commence par la balise

<definitions> et qui contient les balises suivantes :

- <binding> : définit le protocole à utiliser pour invoquer le service web
- <port> : spécifie l'emplacement actuel du service
- <service> : décrit un ensemble de points finaux du réseau

Les opérations possibles et messages sont décrits de façon abstraite mais cette description renvoie à des protocoles réseaux et formats de messages concrets.

et cela quelque soit la technologie utilisée par la fonctionnalité et par l'interface : 'principe du Web'

4 Structure d'un WSDL

Le WSDL HelloService est pris en exemple, il contient ;

Une partie abstraite :

- une opération (opérations),
- avec en entrée un message de type chaîne de caractères (messages, types de données)
- et avec en réponse un message de type chaîne de caractères.

Une partie concrète :

- un lien d'accès,
- un type de transport,
- et un lien de la partie concrète (Binding).

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions targetNamespace="http://demo.com" xmlns:apachesoap="http://xml.apache.org/xml-
soap" xmlns:impl="http://demo.com" xmlns:intf="http://demo.com"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/" xmlns:wsdlsoap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<!--WSDL created by Apache Axis version: 1.4 Built on Apr 22, 2006 (06:55:48 PDT)-->
```

Partie abstraite

```
<wsdl:types>
  <schema elementFormDefault="qualified" targetNamespace="http://demo.com"
xmlns="http://www.w3.org/2001/XMLSchema">
    <element name="helloName">
      <complexType>
        <sequence>
          <element name="name" type="xsd:string"/>
        </sequence>
      </complexType>
    </element>
    <element name="helloNameResponse">
      <complexType>
        <sequence>
          <element name="helloNameReturn" type="xsd:string"/>
        </sequence>
      </complexType>
    </element>
  </schema>
</wsdl:types>

<wsdl:message name="helloNameResponse">
  <wsdl:part element="impl:helloNameResponse" name="parameters">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="helloNameRequest">
  <wsdl:part element="impl:helloName" name="parameters">
  </wsdl:part>
</wsdl:message>

<wsdl:portType name="Hello">
  <wsdl:operation name="helloName">
    <wsdl:input message="impl:helloNameRequest" name="helloNameRequest">
    </wsdl:input>
    <wsdl:output message="impl:helloNameResponse" name="helloNameResponse">
    </wsdl:output>
  </wsdl:operation>
</wsdl:portType>
```

□ de type chaîne de caractères

□ de type chaîne de caractères

□ une seule opération

□ en entrée un nom

□ en réponse un msg

Partie concrète

```
<wsdl:binding name="HelloSoapBinding" type="impl:Hello">
  <wsdlsoap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/> Type de transport SOAP
  <wsdl:operation name="helloName">
    <wsdlsoap:operation soapAction=""/>
    <wsdl:input name="helloNameRequest">
      <wsdlsoap:body use="literal"/>
    </wsdl:input>
    <wsdl:output name="helloNameResponse">
      <wsdlsoap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
</wsdl:binding>

<wsdl:service name="HelloService">
  <wsdl:port binding="impl:HelloSoapBinding" name="Hello">
    <wsdlsoap:address location="http://localhost:8080/hello/services/Hello"/> un lien d'accès
  </wsdl:port>
</wsdl:service>

</wsdl:definitions>
```

5 Principe d'un WSDL

Le WSDL HelloService se décompose suivant 5 fonctionnalités; présenté ci-dessous dans le sens du WSDL, de 5 à 1.

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions targetNamespace="http://demo.com" xmlns:apachesoap="http://xml.apache.org/xml-soap"
xmlns:impl="http://demo.com" xmlns:intf="http://demo.com"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/" xmlns:wsdlsoap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<!--WSDL created by Apache Axis version: 1.4 Built on Apr 22, 2006 (06:55:48 PDT)-->
```

Partie abstraite

5. Types :

La partie Types décrit les types de données utilisés pour le service Web. Il est conforme avec les spécifications XML, définies par le W3C.

```
<wsdl:types>
<schema elementFormDefault="qualified" targetNamespace="http://demo.com"
xmlns="http://www.w3.org/2001/XMLSchema">
  <element name="helloName">
    <complexType>
      <sequence>
        <element name="name" type="xsd:string"/>
      </sequence>
    </complexType>
  </element>
  <element name="helloNameResponse">
    <complexType>
      <sequence>
        <element name="helloNameReturn" type="xsd:string"/>
      </sequence>
    </complexType>
  </element>
</schema>
</wsdl:types>
```

[] de type chaîne de caractères

[] de type chaîne de caractères

4. Message et part (=partie) :

La partie Message décrit le chargement des messages xml qui peuvent être des Réponses, des requêtes et des erreurs. Ses éléments sont définis dans la partie Types.

Le Message est une section donnant le contenu échangé (requête, réponse, ou erreur).

```
<wsdl:message name="helloNameResponse">
  <wsdl:part element="impl:helloNameResponse" name="parameters">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="helloNameRequest">
  <wsdl:part element="impl:helloName" name="parameters">
  </wsdl:part>
</wsdl:message>
```

3. portType et Opération :

La partie PortType définit l'interface de ServiceWeb qui est composé par une/des opérations avec une/des entrées (input), une/des sorties (output). L'opération utilise les messages décrits dans la partie Message.

Le portType est un ensemble abstrait d'opérations accédées via plusieurs Endpoints. (interface)

Le Port ou le point d'accès définit par un URI (adresse d'accès pour un protocole donné) et un Binding.

L'opération (proche d'une méthode au sens Java) est une action proposée par un serviceWeb, décrite par ses messages.

```
<wsdl:portType name="Hello">
  <wsdl:operation name="helloName"> □ une seule opération
    <wsdl:input message="impl:helloNameRequest" name="helloNameRequest"> □ en entrée un nom
    </wsdl:input>
    <wsdl:output message="impl:helloNameResponse" name="helloNameResponse"> □ en réponse un msg
    </wsdl:output>
  </wsdl:operation>
</wsdl:portType>
```

Partie concrète

2. Binding et opération :

La partie Binding (=lien) est le lieu où le protocole (SOAP/http), le style de message (document) et le type d'encodage (Literal) est défini pour chaque opération de la partie portType.

Le Binding est l'utilisation concrète d'un PortType (interface).

Un Binding est une association entre un PortType et des URI pour les différents protocoles d'accès à ce PortType (exple SOAP/http)

```
<wsdl:binding name="HelloSoapBinding" type="impl:Hello">
  <wsdlsoap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/> □ Type de transport SOAP
  <wsdl:operation name="helloName">
    <wsdlsoap:operation soapAction=""/>
    <wsdl:input name="helloNameRequest">
      <wsdlsoap:body use="literal"/>
    </wsdl:input>
    <wsdl:output name="helloNameResponse">
      <wsdlsoap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
</wsdl:binding>
```

1. Service :

La partie service donne la localisation du Web service vers lequel les messages doivent être envoyés.

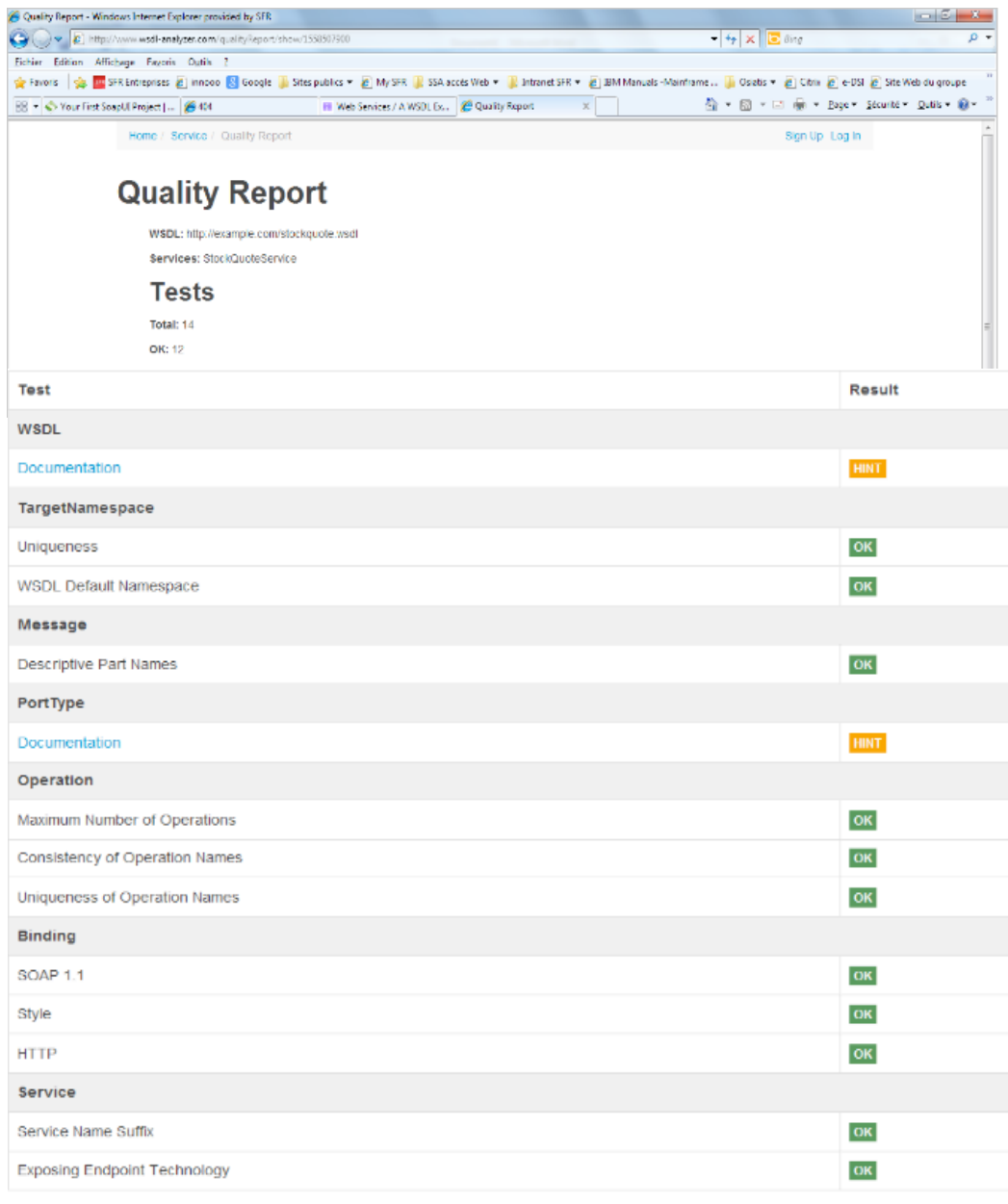
Le service est accessible via une collection de points d'accès ou de ports (endpoints).

```
<wsdl:service>
  <wsdl:port binding="impl:HelloSoapBinding" name="Hello">
    <wsdlsoap:address location="http://localhost:8080/hello/services/Hello"/> □ un lien d'accès
  </wsdl:port>
</wsdl:service>
</wsdl:definitions>
```

6 Tester un WSDL

Nous pouvons utiliser un outil <http://www.wsdl-analyzer.com/> qui nous présente les différentes parties et erreurs.

Nous testons et présentons ici le service StockQuoteService.



Quality Report - Windows Internet Explorer provided by SFR

http://www.wsdl-analyzer.com/quality/report/show/1558507500

Home / Service / Quality Report

Sign Up Log In

Quality Report

WSDL: http://example.com/stockquote.wsdl

Services: StockQuoteService

Tests

Total: 14

OK: 12

Test	Result
WSDL	
Documentation	HINT
TargetNamespace	
Uniqueness	OK
WSDL Default Namespace	OK
Message	
Descriptive Part Names	OK
PortType	
Documentation	HINT
Operation	
Maximum Number of Operations	OK
Consistency of Operation Names	OK
Uniqueness of Operation Names	OK
Binding	
SOAP 1.1	OK
Style	OK
HTTP	OK
Service	
Service Name Suffix	OK
Exposing Endpoint Technology	OK

7 Tester un Web Service avec le Plugin SoapUI

SoapUI est une application open source permettant le test de web service dans une architecture orientée services (SOA).

Ses fonctionnalités incluent l'inspection des web services, l'invocation, le développement, la simulation, le mocking, les tests fonctionnels, les tests de charge et de conformité.



Note : Le Web Service mocking est un outil très utile dans la batterie des tests.

C'est la solution qui permet de répondre à la question «Comment puis-je créer des tests pour un service Web quand il n'y a pas encore de service Web ?".

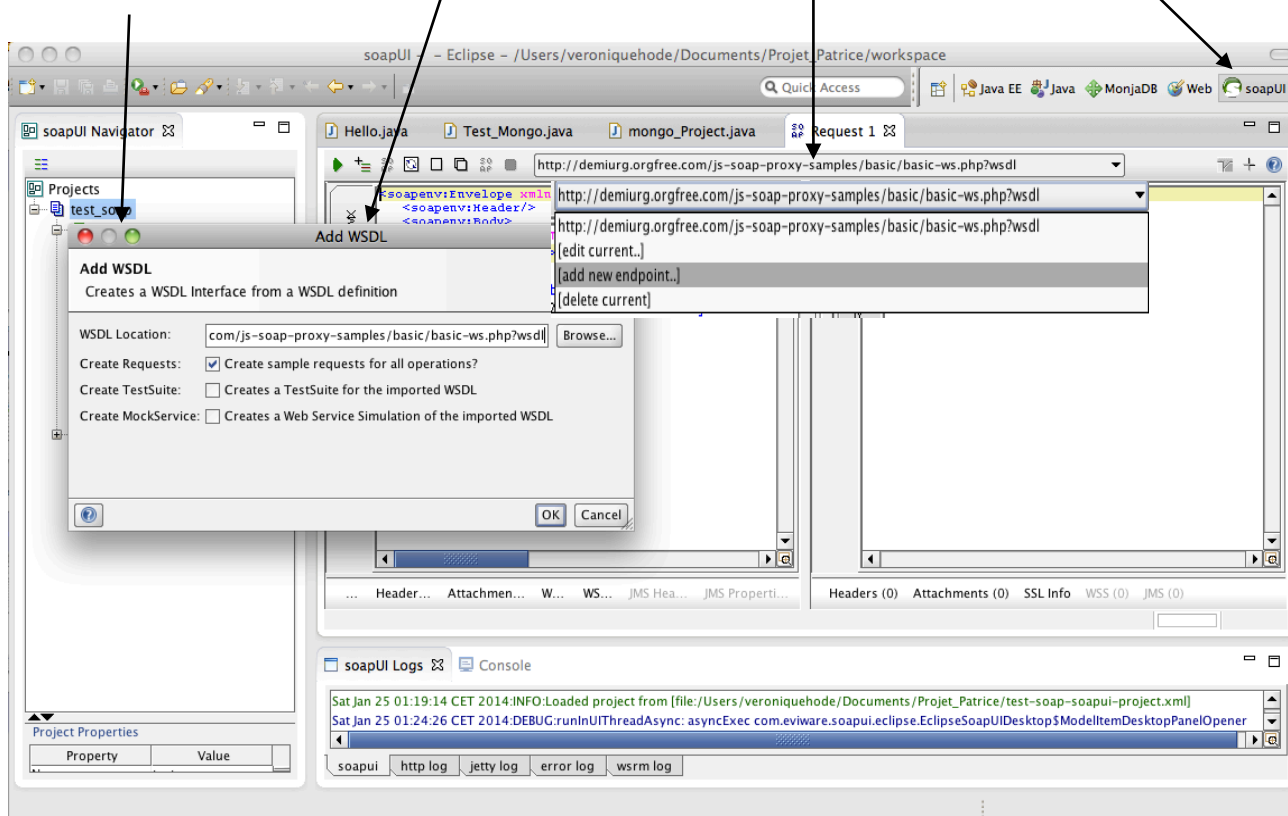
Le mocking permet alors de créer une simulation de la fonction Web avant que le service Web réel soit live.

Nous présenterons ici que l'invocation des services Web.

SoapUI est intégré ici dans un plugin de l'IDE Eclipse Java EE qui est très proche du logiciel du même nom.

Depuis Eclipse on bascule dans une fenêtre SoapUI en sélectionnant la perspective SoapUI.

Il faut ajouter un projet, sélectionner un WSDL et saisir le Endpoint



On saisit le WSDL et le endpoint suivant :

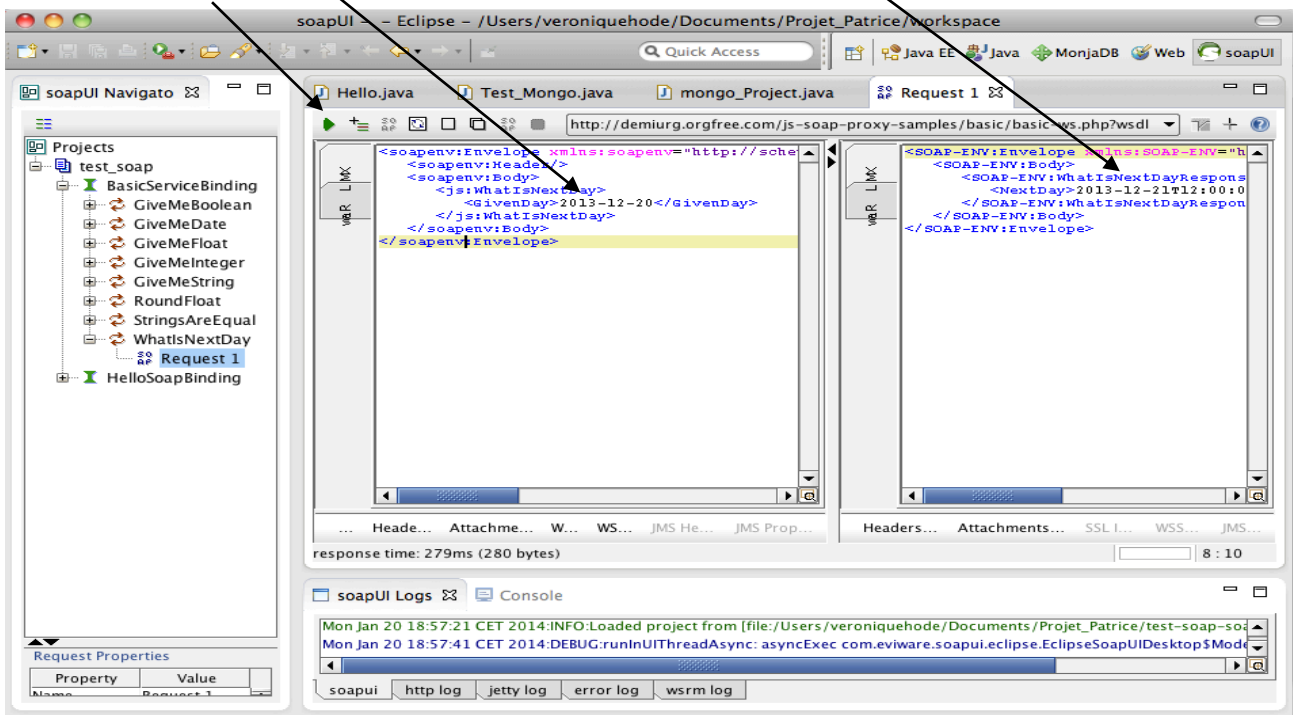
<http://demiurg.orgfree.com/js-soap-proxy-samples/basic/basic-ws.php?wsdl>

On sélectionne l'opération **WhatIsNextDay**

on saisit **2013-12-20** (Client)

après lancement

on obtient le jour suivant **2013-12-21** (Serveur)



SoapUI envoie la requête vers la cible spécifiée et retourne la réponse.

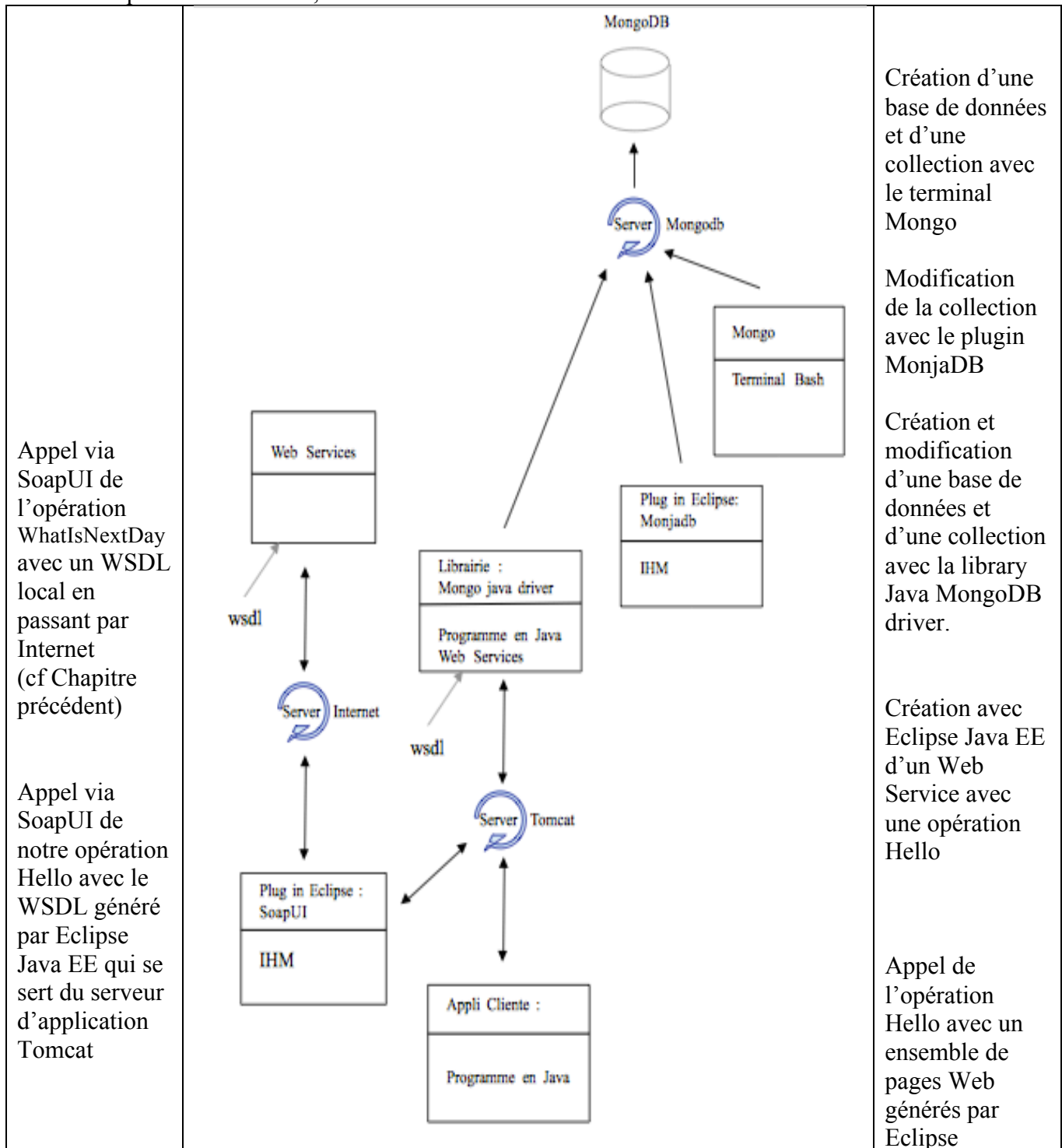
DEUXIEME PARTIE

1 Schéma d'ensemble du projet

Au cours du projet qui suit nous construisons le schéma ci-dessous :

Afin de comprendre les implications de chaque driver, nous les stoppons et faisons des déductions.

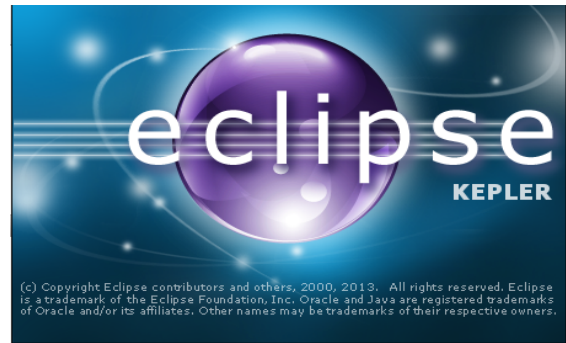
De chaque côtés du dessin, les actions menées.



2 Créer un Web Service avec Eclipse Java EE

Dans cette partie, nous allons créer avec Eclipse et tester SoapUI un simple web service en local.

Pour produire et tester un premier service Web nous allons :



1 Créer un projet dans l'espace de travail Eclipse de type « Dynamic Web Project », qui hébergera le service Web.



2 Ecrire le code Java qui implémente les fonctionnalités du service Web

3 Créer un autre projet de type « Web Service » qui hébergera le client ;
Le serveur d'application Tomcat que l'on utilisera pour accéder et tester le service Web.

Note : Afin de pouvoir créer et exécuter une application comme un service Web en local, nous avons installé la version Eclipse Java EE qui possède le serveur d'application Tomcat.

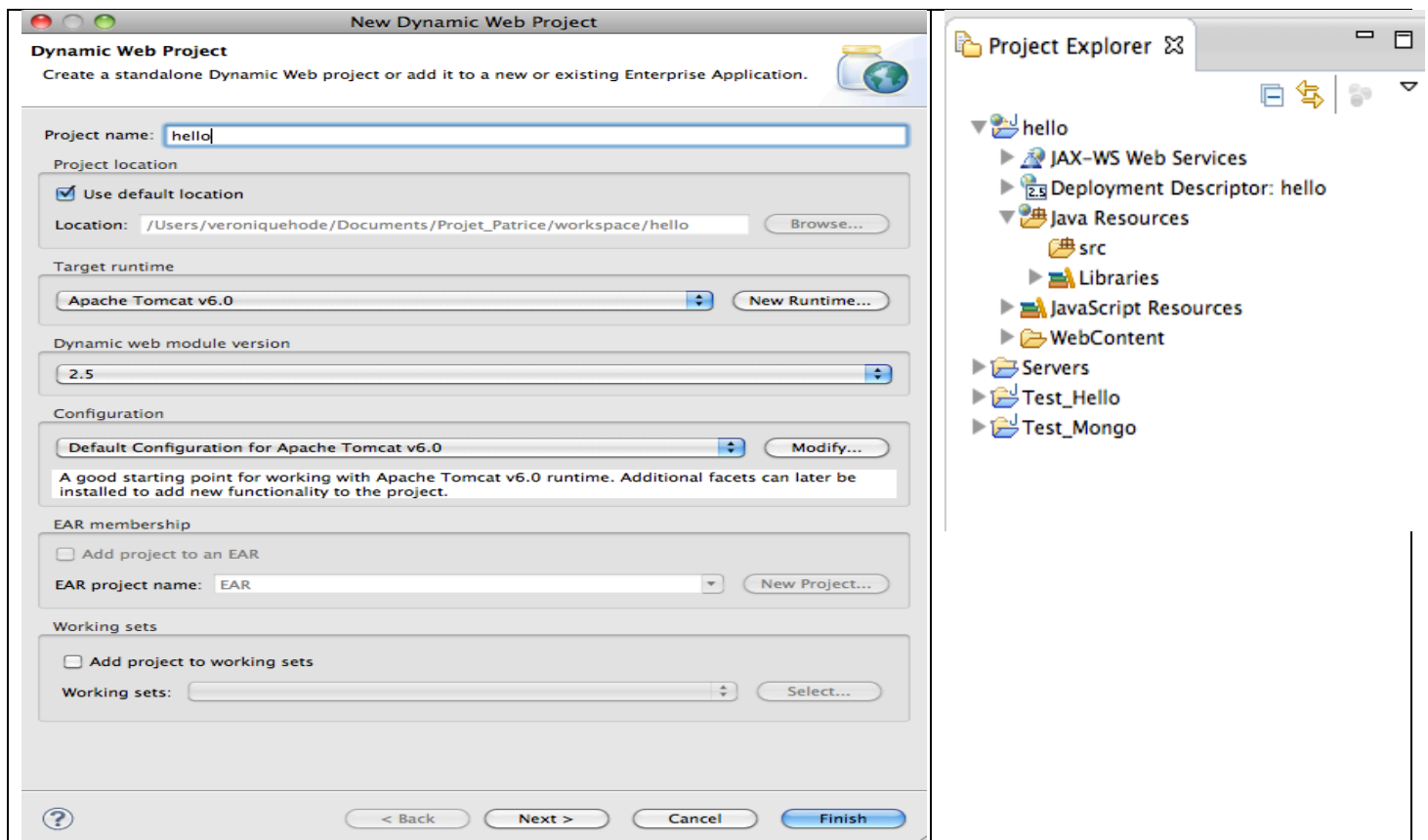
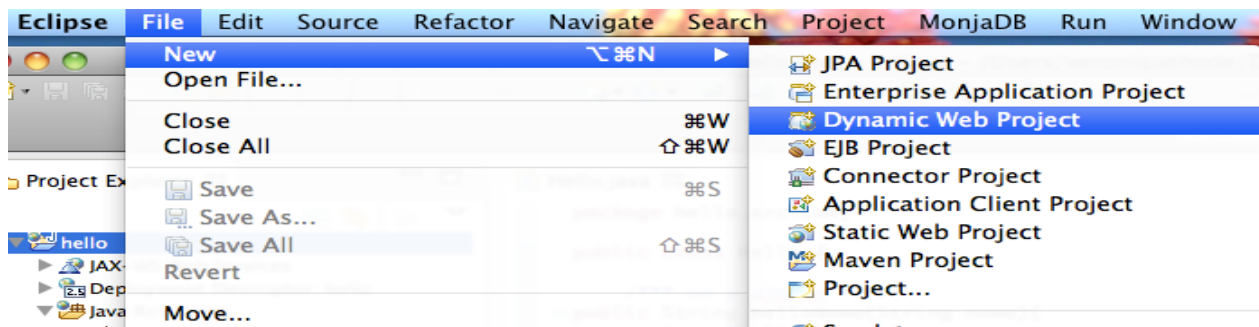
4 Utiliser Eclipse pour générer automatiquement des composants (WSDL) qui transformeront le code Java dans un service Web.

5 Utiliser Eclipse pour générer automatiquement un ensemble de pages web qui fonctionnent en tant que client interface pour appeler le service Web.

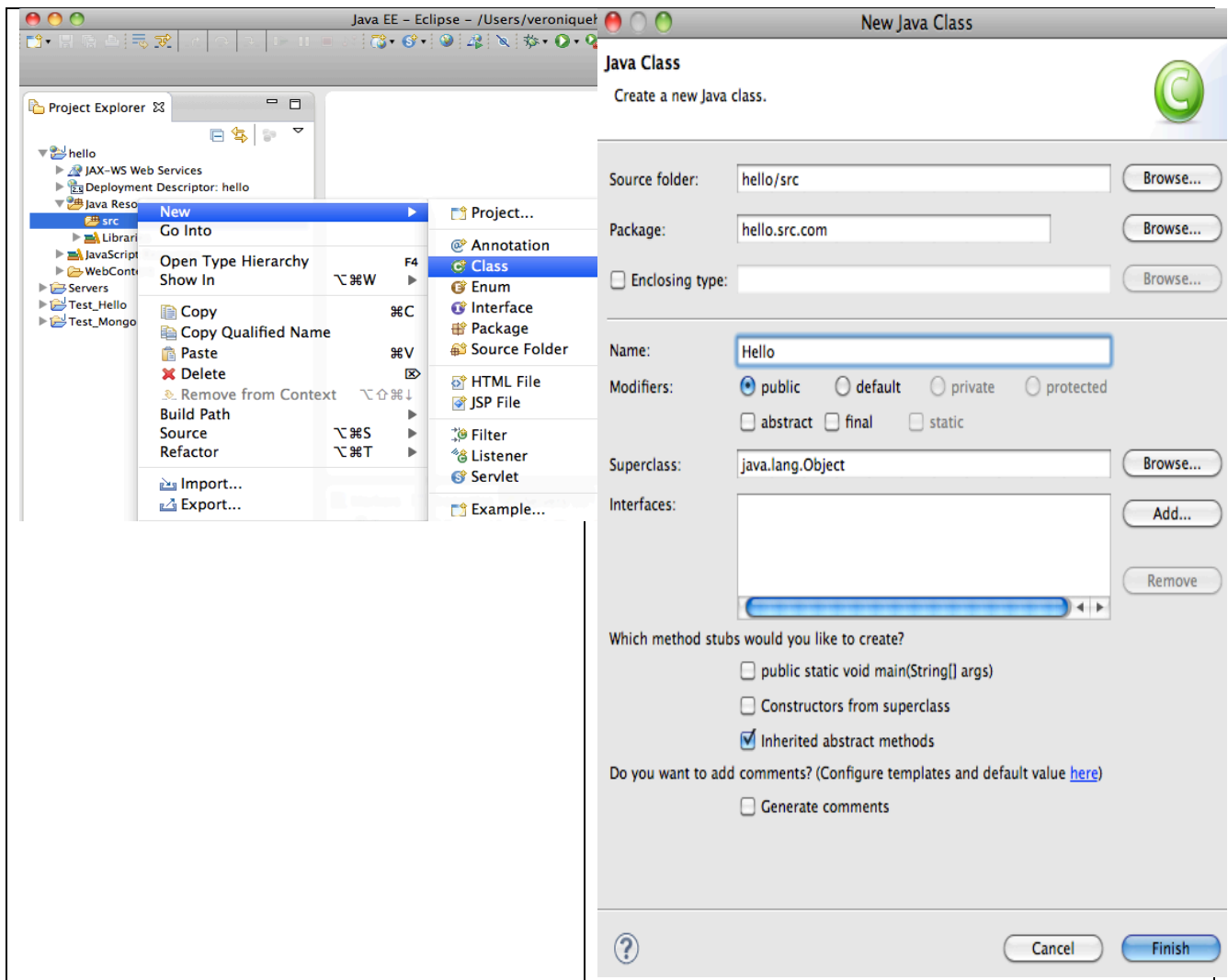
6 Utiliser l'ensemble de pages Web pour envoyer une demande au service Web et observer la réponse de service web.

7 Utiliser l'outil SoapUI pour envoyer une demande au service Web et observer la réponse de service web.

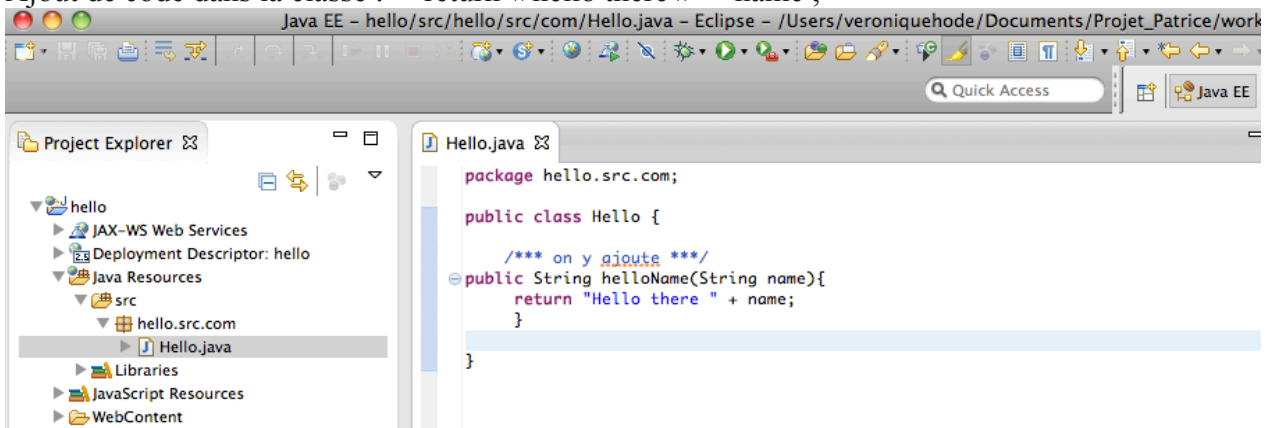
1 Créer un projet dans l'espace de travail Eclipse de type « Dynamic Web Project », qui hébergera le service Web.



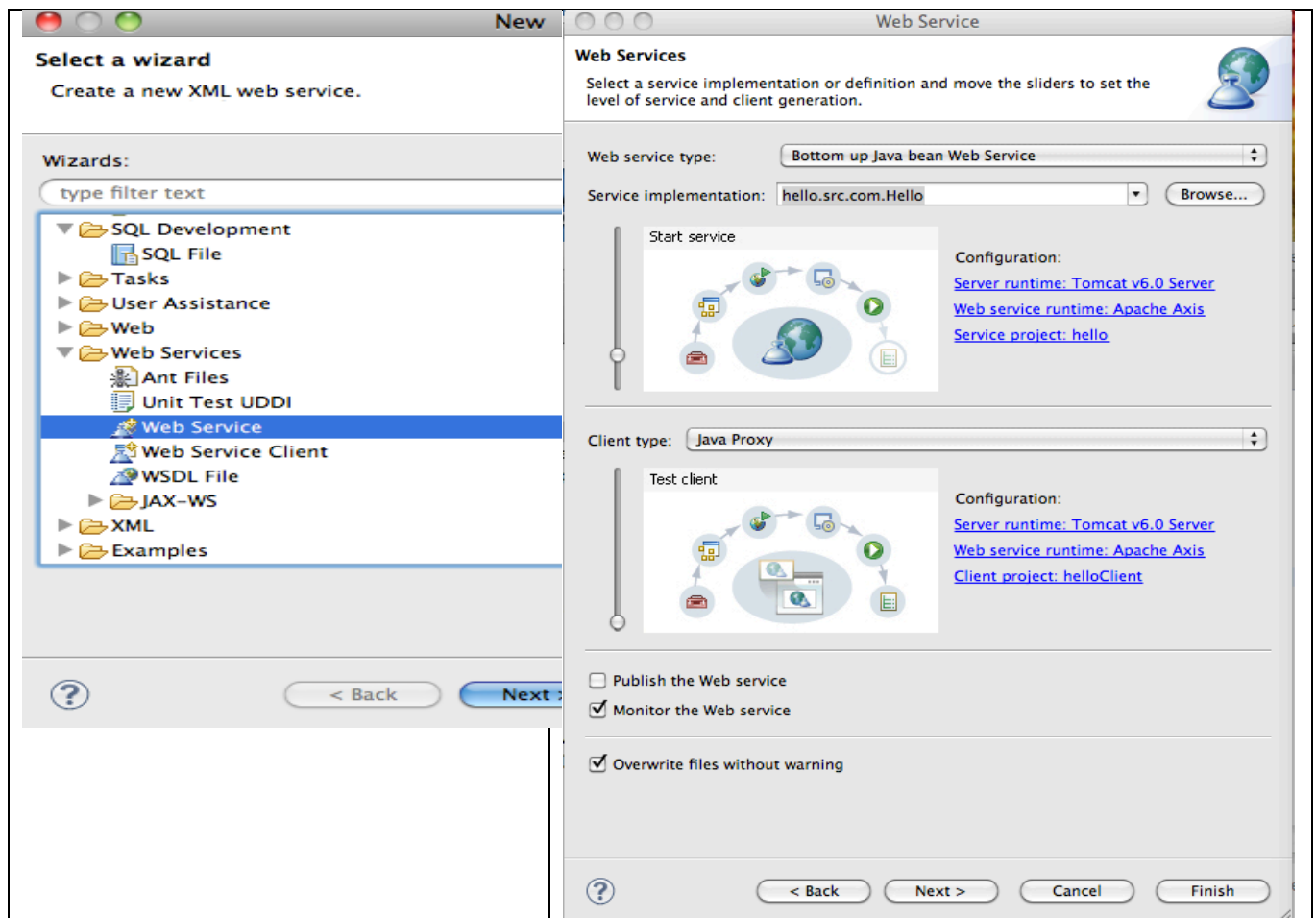
2 Ecrire le code Java qui implémente les fonctionnalités du service Web.



Ajout de code dans la classe : `return « hello there » + name ;`



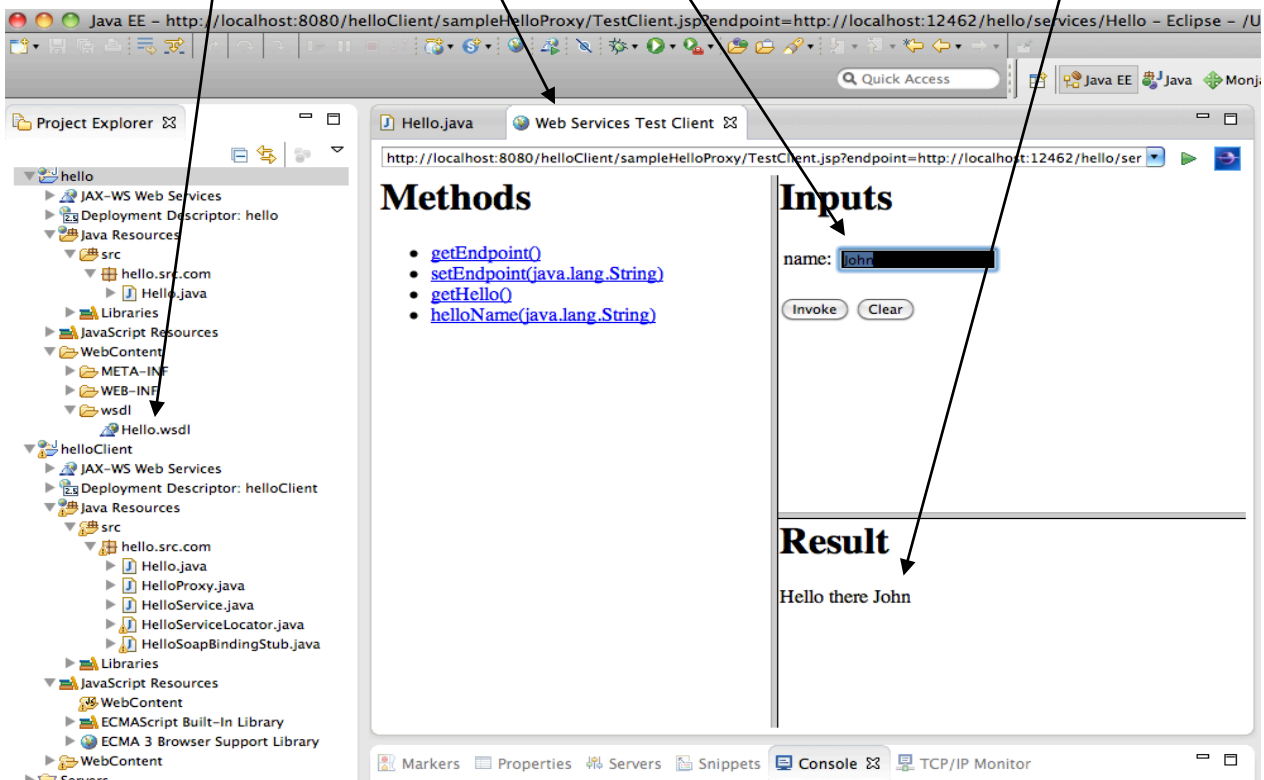
3 Créer un autre projet de type « Web Service» qui hébergera le client ;
Le serveur d'application Tomcat que l'on utilisera pour accéder et tester le service Web.



4 Utiliser Eclipse pour générer automatiquement des composants (WSDL) qui transformeront le code Java dans un service Web.

5 Utiliser Eclipse pour générer automatiquement un ensemble de pages web qui fonctionnent en tant que client interface pour appeler le service Web.

6 Utiliser l'ensemble de page Web pour envoyer une demande au service Web et observer la réponse de service web.

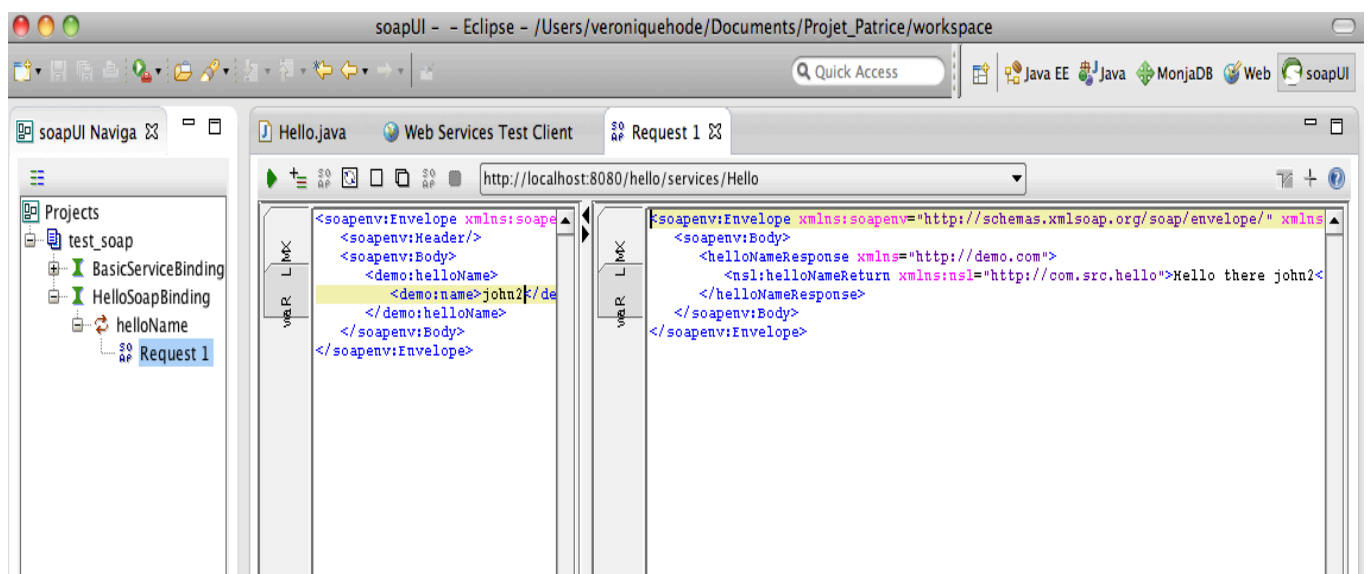
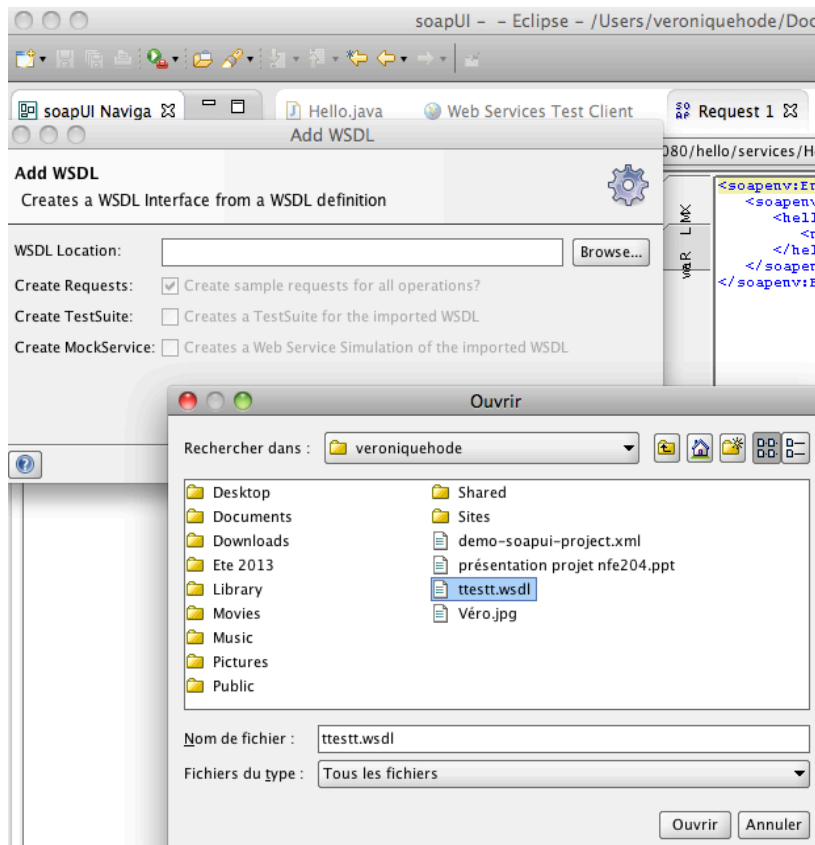


7 Utiliser l'outil SoapUI pour envoyer une demande au service Web et observer la réponse de service web.

Nous copions le WSDL généré par Eclipse dans un répertoire distinct.

Dans SoapUI nous chargeons le WSDL et intégrons le EndPoint en local.

Nous saisissons un nom et lançons la requête pour obtenir le service en réponse.



3 Se connecter à une base MongoDB et ajouter des enregistrements dans une collection



```
Terminal — top — 103x24
Processes: 69 total, 2 running, 67 sleeping, 401 threads
Load Avg: 0.34, 0.23, 0.19 CPU usage: 3.80% user, 4.76% sys, 91.42% idle
SharedLibs: 4920K resident, 4180K data, 0B linkedit.
MemRegions: 13722 total, 984M resident, 26M private, 532M shared.
PhysMem: 573M wired, 1718M active, 1516M inactive, 3808M used, 287M free.
VM: 155G vsize, 1035M framework vsize, 5031618(0) pageins, 810(0) pageouts.
Networks: packets: 3513561/1050M in, 6556115/8450M out. Disks: 274178/12G read, 205301/2993M written.

PID  COMMAND  %CPU  TIME  #TH  #WO  #POR  #MREG  RPRVT  RSHRD  RSIZE  VPRVT  VSIZE  PGRP  PPID
1497  screencap 0.3   00:00.08  2    1    41    80    456K   13M   2820K  12M   2663M  92    1
1489  mongod    0.6   00:01.24  11    0    47    59    28M+   244K  33M+   94M   2636M  1489  1486
1486  bash      0.0   00:00.00  1    0    17    24    384K   856K  1056K  17M   2378M  1486  1485
1485  login     0.0   00:00.01  1    0    22    53    468K   312K  1576K  19M   2379M  1485  1467
1493  mducker   0.0   00:00.00  1    0    50    50    1460K  11M   3840K  31M   2412M  1493  1

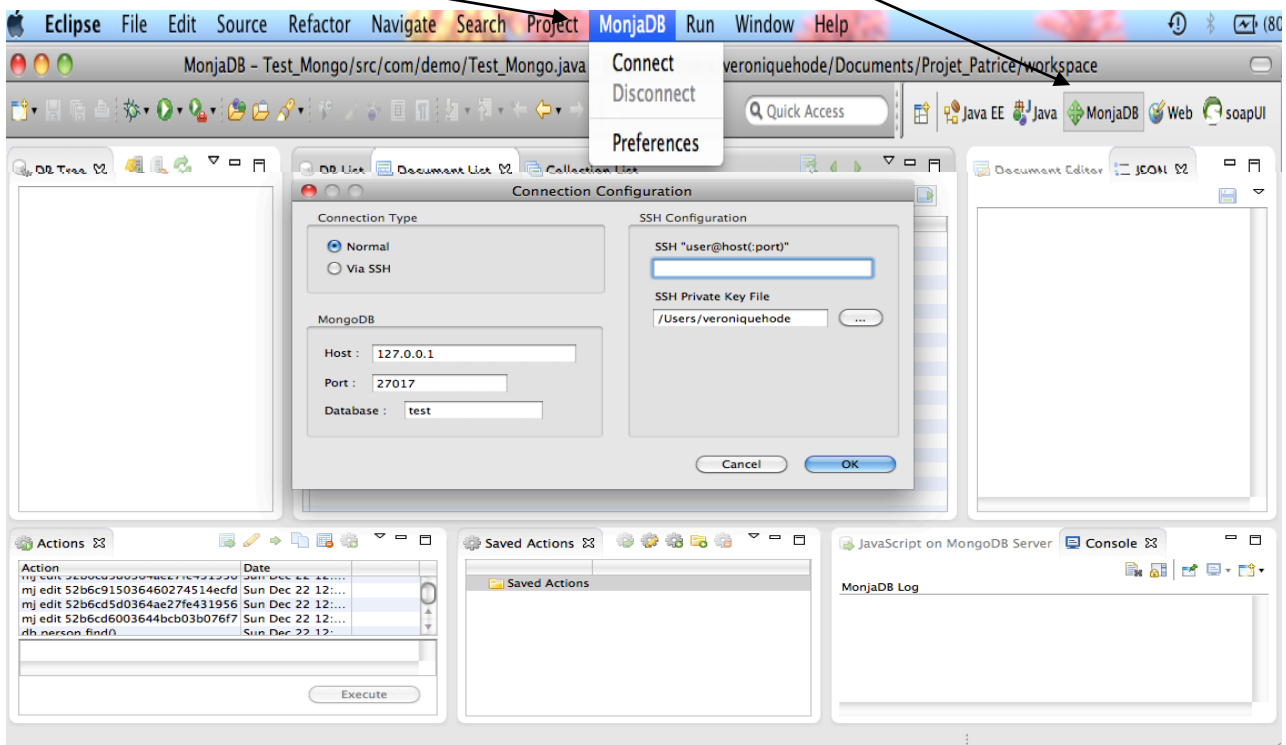
Terminal — mongod — 104x22
Last login: Fri Jan 24 23:57:39 on ttys001
MacBook-Pro-de-veronique-Hode:~ veroniquehode$ /Applications/mongodb/bin/mongod ; exit;
/Applications/mongodb/bin/mongod --help for help and startup options
Fri Jan 24 23:57:55.264 kern.sched unavailable
Fri Jan 24 23:57:55.271 [initandlisten] MongoDB starting : pid=1489 port=27017 dbpath=/data/db/ 64-bit host=MacBook-Pro-de-veronique-Hode.local
Fri Jan 24 23:57:55.271 [initandlisten]
Fri Jan 24 23:57:55.271 [initandlisten] ** WARNING: soft rlimits too low. Number of files is 256, should be at least 1000
Fri Jan 24 23:57:55.271 [initandlisten] db version v2.4.8
Fri Jan 24 23:57:55.271 [initandlisten] git version: a350fc38922fbda2cecd8d5dd842237b904eafc14
Fri Jan 24 23:57:55.271 [initandlisten] build info: Darwin bs-osx-106-x86-64-2.10gen.cc 10.8.0 Darwin Kernel Version 10.8.0: Tue Jun 7 16:32:41 PDT 2011; root:xnu-1504.15.3~1/RELEASE_X86_64 x86_64 BOOST_LIB_VERSION=1_49
Fri Jan 24 23:57:55.271 [initandlisten] allocator: system
Fri Jan 24 23:57:55.271 [initandlisten] options: {}
Fri Jan 24 23:57:55.272 [initandlisten] journal dir=/data/db/journal
Fri Jan 24 23:57:55.272 [initandlisten] recover : no journal files present, no recovery needed
Fri Jan 24 23:57:55.309 [websvr] admin web console waiting for connections on port 28017
Fri Jan 24 23:57:55.309 [initandlisten] waiting for connections on port 27017
```

Voir les commentaires et commandes dans la page ci-dessous.

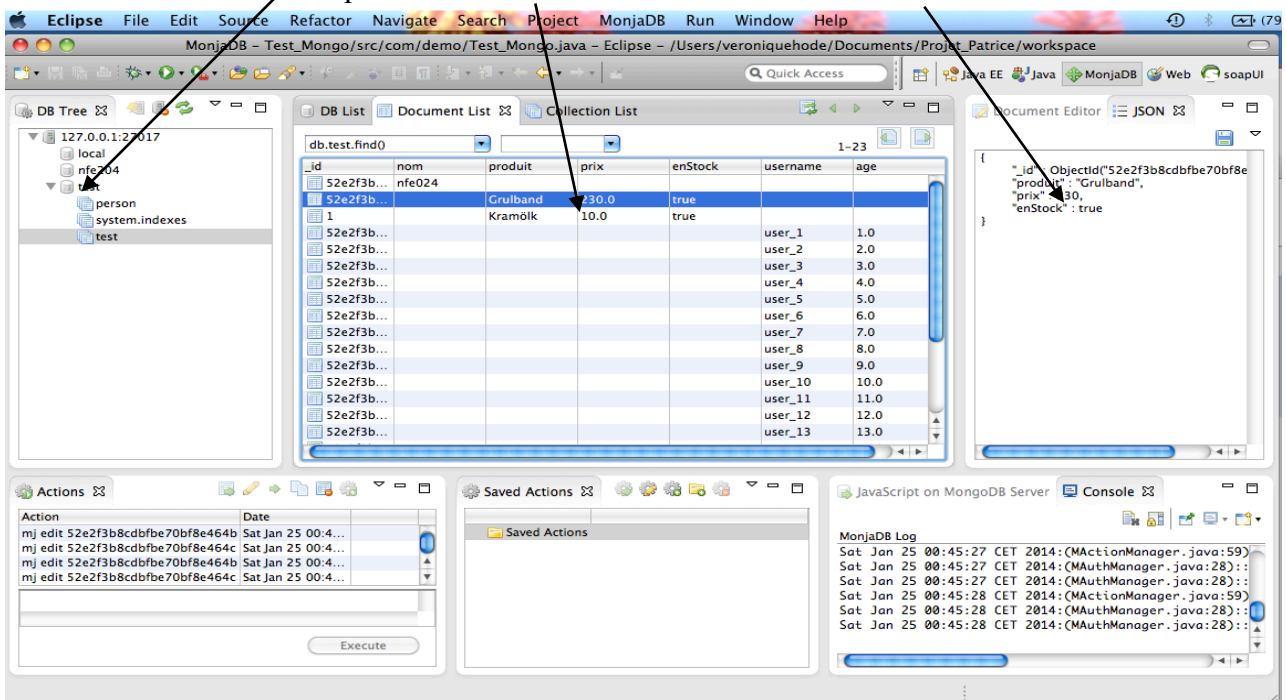
```
Terminal — mongo — 104x46
> // On efface les bases de données
> db.nfe204.remove()
> db.person.remove()
> db.test.remove()
> //on insère
> db.test.insert({"nom": "nfe024"})
> //On affiche le contenu de la collection
> db.test.find()
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e464b"), "nom" : "nfe024" }
> //On insère d'autre document
> db.test.insert({"produit": "Grulband", "prix": 230, "enStock": true})
> //On peut donner un identifiant explicite :
> db.test.insert({"_id": "1", "produit": "Kramölk", "prix": 10, "enStock": true})
> //On peut compter
> db.test.count()
3
> //on peut ajouter des enreg
> for(i = 1; i < 21; i++) db.test.save({ username: 'user_' + i, age: i });
> //On affiche le contenu de la collection
> db.test.find()
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e464b"), "nom" : "nfe024" }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e464c"), "produit" : "Grulband", "prix" : 230, "enStock" : true }
{ "_id" : "1", "produit" : "Kramölk", "prix" : 10, "enStock" : true }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e464d"), "username" : "user_1", "age" : 1 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e464e"), "username" : "user_2", "age" : 2 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e464f"), "username" : "user_3", "age" : 3 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4650"), "username" : "user_4", "age" : 4 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4651"), "username" : "user_5", "age" : 5 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4652"), "username" : "user_6", "age" : 6 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4653"), "username" : "user_7", "age" : 7 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4654"), "username" : "user_8", "age" : 8 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4655"), "username" : "user_9", "age" : 9 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4656"), "username" : "user_10", "age" : 10 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4657"), "username" : "user_11", "age" : 11 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4658"), "username" : "user_12", "age" : 12 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4659"), "username" : "user_13", "age" : 13 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e465a"), "username" : "user_14", "age" : 14 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e465b"), "username" : "user_15", "age" : 15 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e465c"), "username" : "user_16", "age" : 16 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e465d"), "username" : "user_17", "age" : 17 }
Type "it" for more
> it
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e465e"), "username" : "user_18", "age" : 18 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e465f"), "username" : "user_19", "age" : 19 }
{ "_id" : ObjectId("52e2f3b8cdbf70bf8e4660"), "username" : "user_20", "age" : 20 }
>
```

4 Visualiser et modifier une base MongoDB avec le plugin MonjaDB

Afin de créer, manipuler des collections de données, on utilise le plugin Eclipse MonjaDB
Il faut ouvrir la perspective MonjaDB
et se connecter à la base

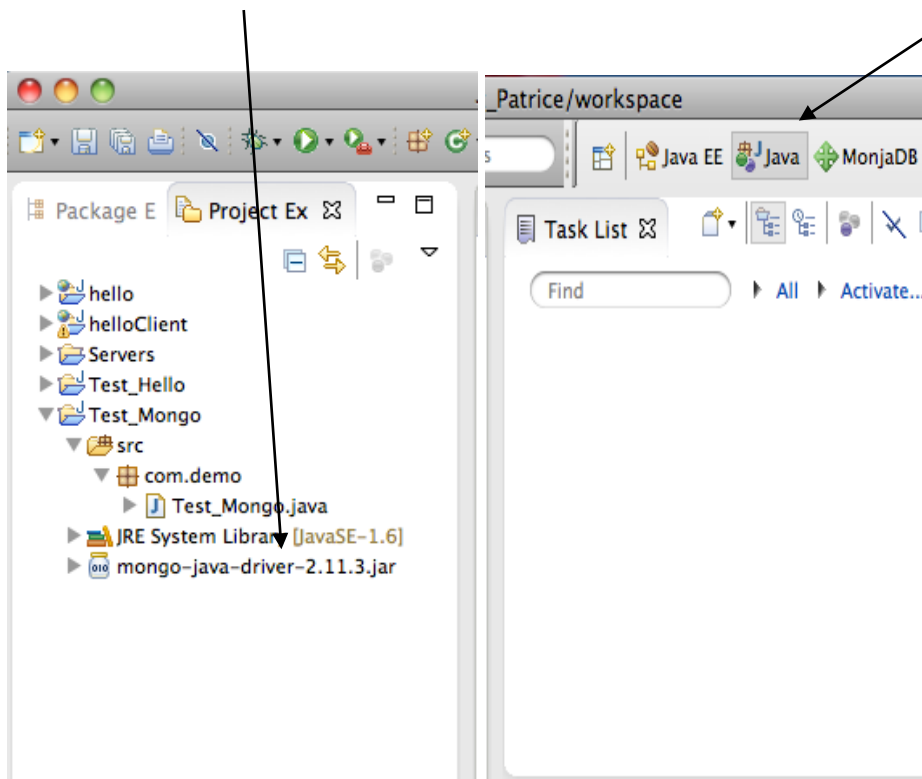


La modification de la base et de la collection test
peut s'effectuer dans les cellules ou dans les valeurs dans le code JSON.



5 Créer des mises à jour avec Java Mongo Driver

A travers un exemple simple nous allons voir comment insérer, afficher, compter et modifier les enregistrements d'une collection en Java. Nous nous connectons en utilisant la perspective Java et la librairie de Java Mongo Driver.



Algorithme du programme

On ajoute 10 enregistrements à la collection, chaque enregistrement contient :

- un nom issu d'un tableau de noms,
- un âge au hasard,
- une date d'arrivée équivalent à la date du jour,
- une adresse prise au hasard issue d'un tableau d'adresse,
- une liste d'amis prise au hasard non redondant.

On affiche ces données

On compte le nombre de personnes qui ont plus de 20 ans

On ajoute à l'âge de John 25 ans

On compte le nombre de personnes qui ont plus de 20 ans

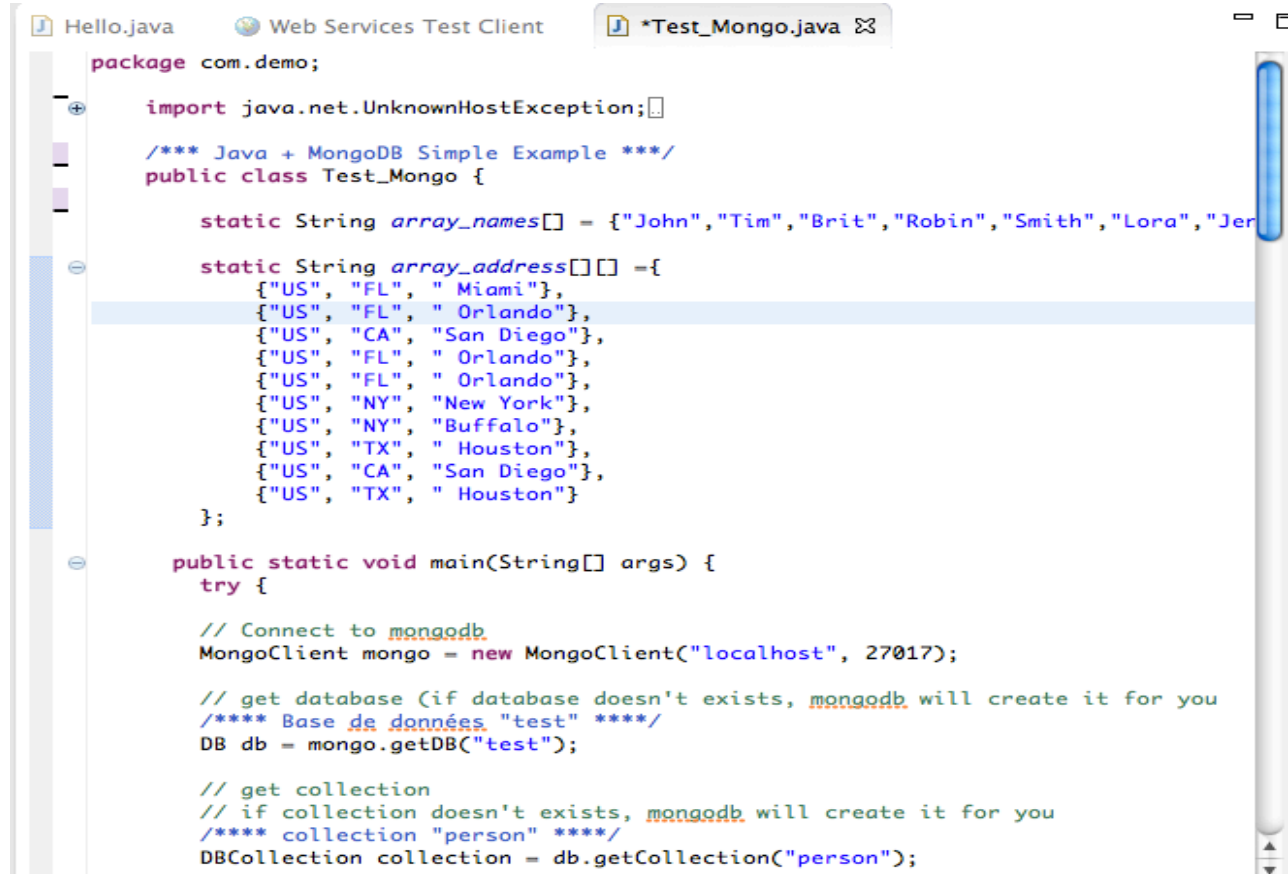
On affiche les nouvelles données

Résultat

```
Nombre de personnes au total : 10
{ "_id" : { "$oid" : "52e58785036474673daf2b48" }, "name" : "Tim", "age" : 18, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "FL", "Miami" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4a" }, "name" : "Robin", "age" : 10, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "FL", "Orlando" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4f" }, "name" : "Victor", "age" : 40, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "CA", "San Diego" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b47" }, "name" : "John", "age" : 16, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "FL", "Orlando" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b49" }, "name" : "Brit", "age" : 53, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "FL", "Orlando" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4b" }, "name" : "Smith", "age" : 48, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "NY", "New York" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4c" }, "name" : "Lora", "age" : 2, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "NY", "Buffalo" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4d" }, "name" : "Jennifer", "age" : 32, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "TX", "Houston" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4e" }, "name" : "Lyla", "age" : 27, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "CA", "San Diego" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b50" }, "name" : "Adam", "age" : 22, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "TX", "Houston" } },
Nb de personne de plus de 20 ans = 6
John prend 25 ans...
Nb de personne de plus de 20 ans = 7
{ "_id" : { "$oid" : "52e58785036474673daf2b48" }, "name" : "Tim", "age" : 18, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "FL", "Miami" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4a" }, "name" : "Robin", "age" : 10, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "FL", "Orlando" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4f" }, "name" : "Victor", "age" : 40, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "CA", "San Diego" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b47" }, "name" : "John", "age" : 41, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "FL", "Orlando" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b49" }, "name" : "Brit", "age" : 53, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "FL", "Orlando" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4b" }, "name" : "Smith", "age" : 48, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "NY", "New York" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4c" }, "name" : "Lora", "age" : 2, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "NY", "Buffalo" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4d" }, "name" : "Jennifer", "age" : 32, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "TX", "Houston" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b4e" }, "name" : "Lyla", "age" : 27, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "CA", "San Diego" } },
{ "_id" : { "$oid" : "52e58785036474673daf2b50" }, "name" : "Adam", "age" : 22, "join" : { "$date" : "2014-01-01" }, "address" : { "US", "TX", "Houston" } },
Done
```

Programme

Déclaration des variables + Connexion à la base



```
package com.demo;

import java.net.UnknownHostException;

/** Java + MongoDB Simple Example */
public class Test_Mongo {

    static String array_names[] = {"John","Tim","Brit","Robin","Smith","Lora","Jennifer","Adam"};

    static String array_address[][] = {
        {"US", "FL", "Miami"},
        {"US", "FL", "Orlando"},
        {"US", "CA", "San Diego"},
        {"US", "FL", "Orlando"},
        {"US", "FL", "Orlando"},
        {"US", "NY", "New York"},
        {"US", "NY", "Buffalo"},
        {"US", "TX", "Houston"},
        {"US", "CA", "San Diego"},
        {"US", "TX", "Houston"}
    };

    public static void main(String[] args) {
        try {
            // Connect to mongodb
            MongoClient mongo = new MongoClient("localhost", 27017);

            // get database (if database doesn't exists, mongodb will create it for you
            /** Base de données "test" ****/
            DB db = mongo.getDB("test");

            // get collection
            // if collection doesn't exists, mongodb will create it for you
            /** collection "person" ****/
            DBCollection collection = db.getCollection("person");
```

Ajout d'enregistrements

```
**** Ajout d'enreg ****/  
// create a document to store key and value  
BasicDBObject document ;  
String address[];  
for(int i = 0 ; i < array_names.length ; i++){  
    document = new BasicDBObject();  
    //value -> String  
    document.append("name", array_names[i]);  
    // value -> int  
    document.append("age", (int)(Math.random()*80));  
    // value -> date  
    document.append("join", new Date());  
  
    address = pickAddress();  
    // value --> document  
    document.append("address", new BasicDBObject("country",address[0])  
        .append("state", address[1])  
        .append("city", address[2]));  
  
    // value -> array  
    document.append("friends", pickFriends());  
  
    collection.insert(document);  
}  
// get count  
System.out.println("Nombre de personnes au total : "+collection.getCount());  
// get all document  
*** on affiche toutes les personnes ***/  
DBCursor cursor = collection.find();  
try {  
    while(cursor.hasNext()) {  
        System.out.println(cursor.next());  
    }  
} finally {  
    cursor.close();  
}  
//-----
```

Sélection et mise à jour

```
// get documents by query  
BasicDBObject query = new BasicDBObject("age", new BasicDBObject("$gt",20));  
  
cursor = collection.find(query);  
System.out.println("Nbre de personne de plus de 20 ans = "+cursor.count());  
  
**** Update -- le temps passe pour John 25 ans plus tard****/  
  
// This example show the use of $inc modifier to increase a particular value. Find doc  
  
BasicDBObject newDocument =  
    new BasicDBObject().append("$inc", new BasicDBObject().append("age", 25));  
  
//collection.update(new BasicDBObject().append("name", "hostB"), newDocument);  
collection.update(new BasicDBObject().append("name", "John"), newDocument);  
  
System.out.println("John prend 25 ans...");  
  
// get documents par un nouveau query  
BasicDBObject query2 = new BasicDBObject("age", new BasicDBObject("$gt",20));  
  
cursor = collection.find(query2);  
System.out.println("Nbre de personne de plus de 20 ans = "+cursor.count());  
  
//get all again  
cursor = collection.find();  
try {  
    while(cursor.hasNext()) {  
        System.out.println(cursor.next());  
    }  
} finally {  
    cursor.close();  
}
```


Suite du résultat des premières données alimentées par les deux procédures ci-après.

```
"address" : { "country" : "US", "state" : "FL", "city" : " Miami", "friends" : [ "Lyla", "John"]}  
· , "address" : { "country" : "US", "state" : "FL", "city" : " Orlando", "friends" : [ "Brit", "Jennifer", "Robin"]}  
} , "address" : { "country" : "US", "state" : "FL", "city" : " Orlando", "friends" : [ "Victor"]}  
· , "address" : { "country" : "US", "state" : "CA", "city" : "San Diego", "friends" : [ "Adam", "Tim", "Lora", "Brit",  
· , "address" : { "country" : "US", "state" : "CA", "city" : "San Diego", "friends" : [ "Victor", "John", "Robin", "Lor  
· , "address" : { "country" : "US", "state" : "NY", "city" : "New York", "friends" : [ "Jennifer"]}  
· , "address" : { "country" : "US", "state" : "FL", "city" : " Orlando", "friends" : [ "Brit", "Victor", "Smith", "Lora"  
'Z'} , "address" : { "country" : "US", "state" : "FL", "city" : " Miami", "friends" : [ "Adam"]  
· , "address" : { "country" : "US", "state" : "FL", "city" : " Orlando", "friends" : [ "John", "Smith"]}  
· , "address" : { "country" : "US", "state" : "FL", "city" : " Orlando", "friends" : [ "Lyla", "Victor", "Jennifer", "A
```

Fin du programme et procédures

```
/* **** Done **** */  
System.out.println("Done");  
  
} catch (UnknownHostException e) {  
    e.printStackTrace();  
} catch (MongoException e) {  
    e.printStackTrace();  
}  
  
}  
//-----  
//These methods are just used to build random data  
private static String[] pickFriends(){  
    int numberOfFriends = (int) (Math.random()* 10);  
    LinkedList<String> friends = new LinkedList<String>();  
    int random = 0;  
    while(friends.size() < numberOfFriends){  
        random = (int) (Math.random()*10);  
        if(!friends.contains(array_names[random]))  
            friends.add(array_names[random]);  
    }  
    String a[] = {};  
    return friends.toArray(a);  
}  
  
private static String[] pickAddress(){  
    int random = (int) (Math.random()*10);  
    return array_address[random];  
}  
}
```

CONCLUSION

Ce projet m'a permis :

D'installer (plusieurs fois) et de faire fonctionner sous Mac : Eclipse EE et des plugins SoapUI et MonjaDB, MongoDB et la librairie Java Mongo Driver.

De développer en Java et de faire des tests en local via SoapUI en utilisant le serveur d'application Tomcat,

De comprendre, de tester et de modifier un programme interrogeant une base de données MongoDB,

D'appréhender l'architecture d'un Web service.

La prochaine étape serait de concevoir un Web Service qui permettrait de saisir un nom et de faire une entrée de ce nom (avec une date d'ajout, un âge, une adresse et ses amis) dans la base de données MongoDB.

De publier ce Web service sur Internet pour pouvoir l'appeler en distant via SoapUI voire d'une application cliente dédiée.

Bibliographie

<http://fr.wikipedia.org/wiki/SoapUI>

<http://www.soapui.org/>

<http://www.soapui.org/Getting-Started/mock-services.html>

<https://www.eclipse.org/>

<http://tomcat.apache.org/>

http://www.eclipse.org/webtools/community/education/web/t320/Implementing_a_Simple_Web_Service.pdf

www.mongodb.org/

<http://tuts.syrinxoon.net/tuts/installation-et-bases-de-mongodb>

<http://hmkcode.com/mongodb-java-simple-example/>

Outils et technologies utilisés

JDK 1.6

Eclipse eclipse-jee-kepler-SR1-macosx-cocoa-x86_64

Plugin Program SoapUI 4.0.1

Server Apache Tomcat 6.0.37

MongoDB 2.4.8

Plugin MonjaDB 1.0.2

Library mongo-java-driver-2.11.3